EAS550 - Data Models Query Language
Performance Tuning Report

Team Members: Aayush Nepal, Emmanuel Joseph, Joebin Peter Soosairaj, Marappa Reddy Churnika

Query Profiled: Monthly Cohort Retention Analysis
File: queries/cohort_retention.sql

This is the most complex of our three analytical queries, featuring:

- 4 CTEs: customer_first_purchase, customer_monthly_activity, cohort_activity, cohort_sizes
- Window functions: LAG, ROW_NUMBER
- Date arithmetic with EXTRACT and DATE_TRUNC
- Multiple GROUP BY aggregations and joins

Baseline Performance (Before Indexing)

Planning Time: 0.816 ms
Execution Time: 842.984 ms

Key Observations from EXPLAIN ANALYZE

Operation	**Time (ms)**	**Notes**
CTE customer_first_purchase	~278	GroupAggregate with Parallel Hash Join
Merge Join (activity × cohort)	~300	Largest bottleneck: cohort-to-activity join with month range filtering
External Merge Sort (disk)	~14	Disk-based sorts due to memory limits
WindowAgg (LAG, ROW_NUMBER)	~48	Efficient incremental sort
Total	~843	Full query execution

Bottlenecks Identified

1. **Parallel sequential scans on orders:** Scanning the orders table by filtering *order_status = 'delivered'* occurs repeatedly.
2. **Customer-order join:** Joining orders that are marked as delivered with the customer IDs increases cost.
3. **External merge sort:** Memory constraints lead to spilling of some sorting processes.
4. **Common table expression (CTE) reuse/materialization cost:** The cohort process is used multiple times downstream.

Indexes Created

```
-- Index 1: Helps delivered-order filtering with timestamp access
CREATE INDEX idx_orders_status_timestamp
ON orders(order_status, purchase_timestamp);

-- Index 2: Helps joins between orders and customers
CREATE INDEX idx_orders_customer_status
ON orders(customer_id, order_status);

-- Index 3: Helps grouping by customer_unique_id
CREATE INDEX idx_customers_unique_id
ON customers(customer_unique_id);
```

Updated Performance (Validated Final Run)

Planning Time: 12.177 ms
Execution Time: 823.717 ms

Analysis

After refining the final query and validating it again with *EXPLAIN ANALYZE*, the execution time improved from 842.984 ms to 823.717 ms.

This marks a small but definite improvement in performance over the existing data set. The execution plan reveals that the dominant costs remain:

- repeated joining of customers with delivered orders 2

- aggregating and summarizing cohort behavior
- orting in grouped and windowed results
- merging externally for intermediate results

Although indexing is applied, there remains significant sequential scanning and sorting by PostgreSQL due to a high proportion of delivered orders. Nonetheless, the optimized query does outperform the previous benchmark.

Why Improvement Is Limited

- The database is not large enough to make PostgreSQL perform better by using more efficient methods in some stages of the process.
- There are several phases of aggregation and sorting that are still costly operations.
- The process of dividing users into cohorts is by nature an iterative process.

Additional Optimizations Considered

1. Increase work_mem

Some sort operations spill to disk. Increasing session memory can reduce external merge sorting:

```
SET work_mem = '16MB';
```

Expected impact: fewer disk-based sort operations and slightly lower execution time.

2. Materialized View

For repeated dashboard or reporting use, the cohort results can be precomputed:

```
CREATE MATERIALIZED VIEW mv_cohort_retention AS
<query>
WITH DATA;

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_cohort_retention;
```

3. Pre-aggregate First Purchase

On a larger scale, precalculating the month of each customer's first order as a new table will save cost from aggregation.

Summary

Metric	**Value**
Baseline Execution Time	842.984 ms
Updated Execution Time	823.717 ms
Improvement	19.267 ms faster (~2.29%)
Indexes Created	3
Primary Bottlenecks	joins, grouping, sorting, external merge operations
Recommended Next Fix	increase work_mem and consider pre-aggregation or materialized views
Dataset Size	~100k orders, ~99k customers, ~112k order items
Validation Method	EXPLAIN ANALYZE on the final cohort retention query